

AgentHub 多 Agent 协作平台研究报告草案

执行摘要

这份 PDF 的核心任务，不是单纯做一个“能聊天”的页面，而是做一个以 IM 会话为前台、以多 Agent 编排为中台、以代码/网页/文件/部署结果为产物层的协作平台。按照用户提供 PDF 《AgentHub-多Agent协作平台设计》第 1-3 页，题目明确要求：IM 式单聊与群聊、多会话并行、完整上下文、主 Agent 协调器、统一适配层、至少接入 2 个主流 Agent 平台、支持用户自建 Agent、产物内联预览；而 Diff、版本历史、部署、多端支持中的部分能力被标为 P2。换句话说，这题真正拿分的主轴，是多 Agent 协作是否跑通、体验是否完整、以及协作规范能否沉淀出来。来源说明：本段及后续“PDF 明确要求”均依据用户提供 PDF 第 1-3 页整理；由于该 PDF 在当前会话中无法通过 `file_search` 直接索引，报告中统一用“用户提供 PDF，第 1-3 页”标注。

如果只给 12 天，最稳的策略不是全面铺开，而是 Web-first，把主链路做深做透。推荐优先级是：会话列表与 IM 主流程、双 Adapter 接入、群聊编排、基础产物卡片、自建 Agent、协作规范资产化（`specs/`、`skills/`、`rules/`）。这样最贴合题面里的高权重考察点，也最适合三人团队在短周期内拿到一个“能演示、能解释、能继续做”的版本。压力不要给太大，先把最值分的部分打穿，P2 功能再保留接口与演示占位即可。

从工程上看，我最推荐的主路线是：Next.js 前端壳体 + FastAPI WebSocket 会话服务 + LangGraph 编排 + LiteLLM 模型网关 + Postgres/Supabase 存储与权限 + 会话级 git worktree + 受控 sandbox + Monaco 编辑器。这条路线的原因很直接：LangGraph 官方把自己定位为长生命周期、状态化 Agent 的 orchestration runtime，强调 durable execution、human-in-the-loop、streaming、memory；FastAPI 官方有清晰的 WebSocket 支持；LiteLLM 提供统一的 LLM gateway；Git 官方 `git worktree` 能让同一个仓库同时挂多个工作树；Supabase Storage 文档明确支持基于 Postgres RLS 的访问控制；Vercel 原生支持 Preview 和 Production 环境分离。把这些拼起来，刚好对上题面需要的会话并行、群聊分工、产物预览、部署卡片和安全边界。 ¹

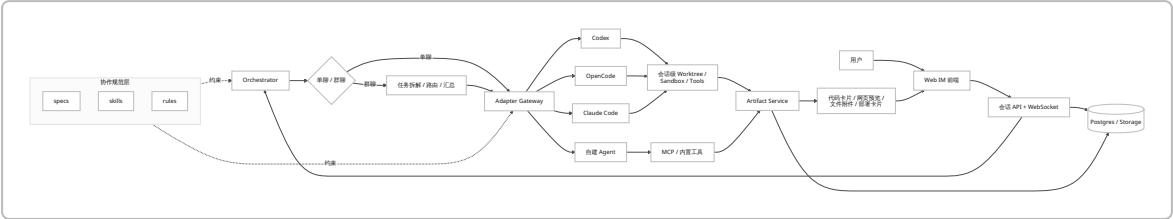
如果团队更偏 OpenAI / Codex 生态，或者特别在意 handoff、guardrails、sandbox 工作空间，那么 OpenAI Agents SDK 路线会更顺手；如果你想把“编排治理、可观测性、checkpointing、time-travel”讲得更体系化，那么 Microsoft Agent Framework 路线会更像“架构型答案”。官方文档显示，OpenAI Agents SDK 原生提供 handoffs、guardrails、sandbox agents、sessions、human-in-the-loop、tracing，而 MAF 官方强调 graph-based multi-agent workflows、checkpointing、streaming、human-in-the-loop、OpenTelemetry observability。 ²

在学术与方法层面，这类平台最值得吸收的不是“某一家模型多强”，而是几个稳定的方法论：ReAct 说明复杂任务需要把 reasoning 与 acting 交替组织；CAMEL 和 AutoGen 说明多角色会话与多 Agent 协作是合理抽象；MetaGPT 进一步把 SOP 和角色职责引入多 Agent 软件工程流程。这与 PDF 强调的“Spec、skill、rules 等协作规范”非常一致，所以规范资产化本身就是交付物的一部分，而不是只放在口头答辩里。 ³

题目要求与边界

先把边界讲清楚，会让后面的技术决策更轻松。你可以把这题拆成四层：会话层、编排层、工作区层、产物层。会话层对应 IM 体验；编排层对应 Orchestrator 与群聊协作；工作区层对应实际的文件、代码、命令、sandbox 和 worktree；产物层对应 Diff、预览、附件、部署结果。这个分层非常重要，因为它能帮助你避免“前端像聊天，后端像脚本，产物像临时文件”的碎片化实现。来源：用户提供 PDF，第 1-3 页。

下面这张图，可以直接当成你们的系统总图使用。它不是唯一方案，但它非常贴题，也适合 12 天排期内压缩实现。



题目中的**明确要求**、**P2 项与未指定项**，建议在开工第一天就固化成追踪矩阵。下面这张表可以直接拿去变成 docs/requirements-traceability.md。

类别	内容	结论
PDF 明确要求	IM 聊天式交互、对话列表、单聊、群聊、上下文连续、产物内联、主 Agent 协调器、统一适配器层、至少接入 2 个主流 Agent 平台、支持自建 Agent、实时预览与二次编辑	必须进入 MVP 。来源：用户提供 PDF，第 1-3 页
PDF 标注 P2	PPT 浏览、Diff 视图、版本历史、对话式局部修改、部署发布、多端支持的完整实现	本期做接口与可演示最小版，不追求全部完成 。来源：用户提供 PDF，第 1-3 页
PDF 考察点	AI 协作能力、功能完整度、生成效果质量、代码理解度、创新与产品感；并特别点名 Spec/skill/rules	必须把规范资产化，不能只做 UI 。来源：用户提供 PDF，第 1-3 页
PDF 交付物	产品设计文档、技术文档、可运行 Demo、AI 协作开发记录、3 分钟 Demo 视频	最后两天必须留给包装与录制 。来源：用户提供 PDF，第 1-3 页
PDF 未指定	预算、确切工期长度、部署环境、并发规模、验收环境、必须接入哪两个平台、模型预算上限	统一标注“未指定”并给出默认假设

下面这些假设，是为了让方案可执行，不是把题意改掉。

未指定项	状态	默认假设	说明
预算	未指定	课程 / 毕设 Demo 级预算	只估算基础托管量级，模型费用不写死
工期	未指定	12 个工作日	按用户当前约束执行
起始日	未指定	2026-05-25	按用户要求，若改起始日则整体顺延工作日
部署环境	未指定	Web-first, 1 个开发环境 + 1 个预览环境	桌面/移动不做本期主实现
接入平台	未指定	默认 Codex + OpenCode ；若 Anthropic 账号就绪，可换成 Codex + Claude Code 或三者都接	题面示例包含 Claude Code、Codex、OpenCode

未指定项	状态	默认假设	说明
用户规模	未指定	50-100 DAU、5-10 个并发 Agent 任务	足够支撑课程演示与小范围试用
产物类型	部分指定	首期聚焦网页 / 代码 / 文本 / 文件附件	PPT、复杂文档渲染放入 P2

从工程角度，一个很关键但容易被忽略的点是：**规范资产需要“落盘”**。Claude Code 官方文档明确支持 `CLAUDE.md`、`skills`、`hooks`、`MCP`、自定义代理和并行代理；OpenCode 官方把 `build` / `plan` Agent、权限、`@` 调用、`SKILL.md` 都做成正式概念；MCP 规范本身把 `resources`、`prompts`、`tools` 都标准化了。这意味着你完全可以把“团队如何与 AI 协作”做成实际目录结构，而不是 PPT 里的口号。 4

推荐的仓库骨架如下：

```
apps/web/
services/api/
services/orchestrator/
services/adapters/
services/runtime/
services/artifacts/
services/deploy/
packages/protocol/
packages/ui/
specs/
skills/
rules/
agents/
docs/
demo/
```

推荐技术路径

先说判断标准。对这题来说，路线优劣不在于“谁最先进”，而在于三件事：**是否贴合题面、三人能否 12 天内交付、答辩能否讲清楚**。如果一条路线非常强，但需要很多底层治理与运维投入，它未必适合当前约束。反过来，如果一条路线能快速做出会话主链路、群聊编排和产物卡片，那就是更优路线。压力不要给太大，先把高分主链路做完整，比“全栈全端全做”要稳得多。

同时，这题天然适合 **orchestrator-worker** 模式。LangGraph 官方在工作流文档中单独列出 `orchestrator-worker`、`parallelization`、`routing` 等模式；OpenAI Agents SDK 也把 `handoffs`、`sessions`、`sandbox` agents、`guardrails` 作为核心原语；MAF 则把 `group collaboration`、`sequential`、`concurrent`、`handoff`、`checkpointing` 列为核心工作流能力。结合 ReAct、AutoGen、CAMEL、MetaGPT，你可以理直气壮地把“多 Agent 群聊”实现为**有状态的图编排 + 受控工具调用 + 结构化产物回传**。 5

技术路径对比表

路线	建议组件	优点	代价 / 缺点	可扩展性	安全性	成本估算	依据
路线 A Web-first 平衡型 推荐	Next.js + FastAPI WebSocket + LangGraph + LiteLLM + Postgres / Storage + git worktree + Monaco	对题面最贴；前后端分工清晰；能较快做出会话流、群聊编排、产物预览；可解释性强	需要自己定义消息协议、artifact schema、adapter contract；需要一定后端整合能力	中高。可从课程 Demo 平滑扩到小规模产品	中高。可用会话级 worktree 隔离、RLS、私有 bucket、Preview/Prod 分离、命令审批	预算未指定；若走 Vercel Pro，起步为 \$20/月 + usage；其余数据库/对象存储/模型费用按供应商实时计费，本文记为未指定。	7
路线 B OpenAI / Codex 强化型	OpenAI Agents SDK + Codex + Web 前端 + DB / Storage	handoffs、guardrails、sessions、sandbox agents、MCP、tracing 都是正式能力；很适合“编码代理 + 审批 + 真实工作区”	更偏 OpenAI / Codex 生态；跨供应商虽可做，但抽象重点在 OpenAI 体系	中高	高。官方原生有 guardrails、HITL、sandbox、approval 语义	预算未指定；在默认假设下，因为 sandbox / tracing / 审批流更重，通常比路线 A 的实现与运维成本更高，这是工程推断。	8
路线 C 治理与 workflow 强化型	Microsoft Agent Framework + Web 前端 + 运行时与观测链路	最容易讲清楚架构；对 graph workflow、checkpointing、streaming、observability 支持完整；适合把 Orchestrator 做得很“像系统”	学习曲线更陡；12 天内若没有既有经验，容易把时间花在框架理解而不是用户体验	高	高。OpenTelemetry、durability、human-in-the-loop 是强项	预算未指定；在默认假设下，因为多了治理、观测与更重的工作流层，落地成本通常高于路线 A，这是工程推断。	9

我给出的明确建议是：**主线采用路线 A**，同时吸收路线 B 的 sandbox / handoff 思路，以及路线 C 的 checkpointing / observability 思路。理由很简单。这条路线既能快，又能讲；既能稳稳贴题，又不会过度依赖某一家生态。

默认推荐的落地选型

层次	推荐选型	为什么这样选
会话前端	Vercel Chatbot 模板 为壳，重写联系人 / 群聊 / 产物卡片	它已经有 <code>app/</code> 、 <code>artifacts/</code> 、 <code>components/</code> 、 <code>hooks/</code> 、 <code>lib/</code> 等结构，能显著缩短 UI 起步时间。 ¹⁰
实时会话层	FastAPI + WebSocket	IM 体验需要服务端持续推事件；FastAPI 官方有成熟 WebSocket 文档。 ¹¹
编排层	LangGraph	官方把 durable execution、human-in-the-loop、memory、event streaming 都作为核心能力；很适合做 Orchestrator。 ¹²
模型 / Provider 网关	LiteLLM	一个统一接口接 100+ providers，且能以 Python SDK 或 Proxy/Gateway 方式运行。 ¹³
Adapter 目标	默认先接 Codex + OpenCode ，Claude Code 作第三选择	Codex 官方有 subagents、worktrees、approval、MCP；OpenCode 官方有 build/plan、permissions、 <code>SKILL.md</code> 、自定义 agents。两者非常贴题。 ¹⁴
运行时	会话级 <code>git worktree</code> + sandbox	Git 官方支持同仓库多工作树并行，非常适合“一会话一工作区”；Codex 也明确把 thread 与 worktree 关联。 ¹⁵
存储与权限	Postgres + 对象存储；对象访问私有化 bucket + RLS	Supabase Storage 官方明确支持基于 Postgres RLS 的访问控制，且默认不会在没有 RLS 策略时允许上传。 ¹⁶
代码产物编辑	Monaco Editor	浏览器内代码编辑器的成熟选型，适合代码卡片、展开编辑与基础 diff 包装。 ¹⁷
规范层	<code>specs/</code> 、 <code>skills/</code> 、 <code>rules/</code> 、 <code>agents/</code>	这直接对应题目强调的 Spec / skill / rules；OpenCode、Claude Code、MCP 都支持类似思路。 ¹⁸

推荐的安全基线

这题的安全，不要理解成“最后补个鉴权”。真正的安全边界有四个：**工作区隔离、文件权限、工具审批、环境隔离**。

一是工作区隔离。Git 官方 `git worktree` 支持同仓库多个工作树、多个分支同时检出，天然适合“每个会话一个独立工作区”；Codex 的 worktree 文档也说明一个线程会持续绑定到它的 worktree，且默认在受控的 detached HEAD 工作树中运行。 ¹⁵

二是文件权限。Supabase Storage 文档明确写到，Storage 默认不会在没有 RLS 策略时允许上传；策略定义在 `storage.objects` 表上，而且可以把访问限制到具体 bucket、具体私有目录或具体用户目录。 ¹⁹

三是工具审批。OpenCode 官方把 `allow / ask / deny` 做成了正式权限系统，`plan` Agent 默认对 file edits 和 bash 采取更保守的权限；Codex 也把 approvals 和 sandboxing 作为正式功能；OpenAI 的 MCP /

Connectors 文档则建议默认开启 approval，并明确提醒 third-party MCP server 的 prompt injection 与数据暴露风险。 ²⁰

四是环境隔离。Vercel 部署文档明确区分 Local、Preview、Production 三种环境，并支持环境变量按环境管理；这非常适合把“聊天触发部署”限制在 Preview，而不是直接指向 Production。 ²¹

成本口径说明

PDF 没给预算，所以**模型费用、CLI 订阅费用、对象存储增量费用、预览环境构建费用**都应标注为“未指定”。本报告只给最保守、最能核对的基础托管口径。

成本项	口径
Web 托管	若使用 Vercel Pro，官方定价显示为 \$20/月 + additional usage 。 ⁶
数据库 / 对象存储	未指定 。依供应商与实际用量而变；本报告不写死固定数字。
LLM / Agent 平台账号	未指定 。高度取决于接入平台、调用量、是否使用 API 或订阅。
本地开发	可先按本地 / 开发机运行，基础成本可近似看作 未新增托管成本 。
推荐口径	对答辩建议写成：“ 基础基础设施成本可控；模型与调用成本未指定，按实时使用量结算。 ”

可直接采用的成熟 GitHub 项目

这一部分只选**真的能帮你省时间**的项目，不选“研究好看但落地代价很大”的项目。成熟度是我基于仓库结构、许可证、官方文档、功能完备度和适合当前题目的程度作出的工程判断，属于**主观评估**。

项目	简述	许可证	成熟度	关键文件 / 目录	如何改造
LangGraph	多 Agent 编排主干。适合实现 Orchestrator、路由、并发、恢复、人工介入	MIT	高	docs/、examples/、libs/、AGENTS.md、CLAUDE.md	只拿来 做编排 ，不要把 UI 和业务细节全塞进去
LiteLLM	统一模型网关，单接口接 100+ provider；适合做模型 / provider 抽象层	主体 MIT，enterprise/ 目录例外	高	litellm/proxy/proxy_cli.py、model_prices_and_context_window.json、docker-compose.yml	放在 adapter 之前，统一模型路由、日志、预算与限流
OpenAI Agents SDK	强调 handoffs、guardrails、sessions、sandbox agents、tracing 的多 Agent SDK	MIT	高	src/agents/、examples/、.agents/skills/、AGENTS.md	适合借它的 sandbox / handoff / session 模型，也可直接做路线 B 的主干

项目	简述	许可证	成熟度	关键文件 / 目录	如何改造
Microsoft Agent Framework	工作流治理型框架, 适合强调 graph orchestration、checkpointing、observability	MIT	中高	python/、declarative-agents/、schemas/	若团队后端强, 可作为 Orchestrator 替代方案; 否则更适合借鉴工作流与治理设计
OpenCode	开源编码 Agent; 内建 build/plan、自定义 agents、SKILL.md、权限系统	MIT	高	.opencode/、packages/、specs/、sdks/vscode/、README.zh.md	最适合做第二个 adapter, 也非常适合借它的 Agent / Skill / Permission 设计
Vercel Chatbot	可黑可改的 Next.js 聊天壳体; 前端起手非常省时	Apache-2.0	高	app/、artifacts/、components/、hooks/、lib/、.env.example、drizzle.config.ts	保留消息流与页面骨架, 重写联系人、群聊、artifact 卡片与协议绑定
OpenHands	值得借的是 runtime / local GUI / REST API / sandboxes 的抽象, 不建议整仓硬 fork	核心 MIT, enterprise/ 目录例外	高	frontend/、openhands/、skills/、config.template.toml、docker-compose.yml	借鉴 conversations / runtime / sandbox / local GUI 的模块边界
Monaco Editor	浏览器内代码编辑器, 适合代码卡片展开、全屏编辑、基础 diff 封装	MIT	高	src/、samples/、docs/、monaco-lsp-client/、webpack-plugin/	把它接到 artifact panel 中, 不要先追求完整 IDE 体验

还有两类“适合借鉴但不建议作为本期主干”的资源, 也值得一提。第一类是 **Claude Code / Agent Skills / MCP** 这套生态。官方文档已经把 CLAUDE.md、skills、hooks、MCP、多代理、后台代理说得很清楚, 而且还提供 PowerPoint / Excel / Word / PDF 的预构建 Agent Skills, 非常适合拿来支撑“P2 文档类产物”的架构讲述。 ²⁹

第二类是 **MCP 规范本身**。如果要做统一工具接入层, MCP 的价值不是“赶时髦”, 而是它把 Resources、Prompts、Tools、Logging、Progress、Cancellation 和安全原则都标准化了; OpenAI 的官方 MCP/Connectors 文档也已经把 remote MCP server、approval、tool listing、日志审查和 trusted server 建议写得 very 明确。对 AgentHub 这种“多 Agent + 多工具 + 多产物”系统来说, MCP 很适合做中长期的工具抽象底座。

³⁰

团队配置与 12 天排期

在 12 个工作日里, 三人团队最怕的不是任务重, 而是**角色边界不清导致互相等待**。所以建议用“前端主责 / 编排主责 / 平台主责”的分法, 但每个人都要能跨一层帮忙, 不要把某个模块做成单点瓶颈。

团队角色与工时估算

角色	建议定位	核心职责	主要产出	预计工时
角色A	产品交互 / 前端负责人	会话列表、聊天流、群聊交互、产物卡片、Monaco 集成、Demo 叙事	apps/web/*、packages/ui/*、demo/*、产品文档	96h
角色B	AI 编排 / 后端负责人	事件协议、双 Adapter、Orchestrator、路由与汇总、自建 Agent、skills/rules	services/orchestrator/*、services/adapters/*、packages/protocol/*、skills/*	96h
角色C	平台 / 运行时负责人	存储、鉴权、worktree、sandbox、artifact 存储、预览/部署、CI	services/runtime/*、services/artifacts/*、services/deploy/*、db/*	96h

如果你希望工时拆得更细一点，可以这样理解：

角色	模块拆分	预计工时
角色A	IM 壳体 32h；产物卡片与 Monaco 28h；联调与打磨 20h；Demo 与文档 16h	96h
角色B	协议与数据模型 16h；Adapter 双接入 32h；Orchestrator 32h；自建 Agent 与评测脚本 16h	96h
角色C	DB/Auth/Storage 24h；worktree/sandbox 28h；预览/部署 24h；CI/稳定性/回滚 20h	96h

12 个工作日的基线排期

这里采用默认起始日 **2026-05-25**。如果你后续指定了别的起始日，只需要把下面所有日期按**工作日整体平移**，任务顺序不变。



这 12 天里，最关键的时间点只有三个。**第 5 个工作日**之前，必须让“会话 + streaming + 单聊 + 至少一个 adapter”跑通；**第 8 个工作日**之前，必须让“群聊 + Orchestrator + 双 adapter + 汇总”可演示；**第 12 个工作日**之前，必须完成“产物卡片 + 自建 Agent + 文档 + Demo 视频”。如果第 8 天还没看到群聊协作，那就不是小延误，而是范围失控了。

更短 / 更长工期的替代假设

假设	适用情况	裁剪或扩展建议
10 天压缩版	用户时间进一步缩短	砍掉基础 Diff、部署卡片真实执行，只保留“预览 URL 占位 / ZIP 导出”；自建 Agent 只做最小字段
12 天基线版	当前推荐	完成 Web-first 主链路、双 Adapter、群聊、产物卡片、自建 Agent、规范资产与 Demo
15 天舒适版	如果能多争取 3 天	加基础 Diff / Apply、版本历史只读、部署状态回写、简单移动端预览页

阶段检查点与每日产出

这里我分成两张表来看。第一张是**阶段检查点**，用于确保每个阶段都有清晰的“过线标准”；第二张是**12 天的通俗产出清单**，也就是“今天下班前一定要看见什么”。

分阶段检查点

阶段	日期	核心交付物	验收标准	主要风险	缓解措施
阶段一 范围冻结与协议成型	05-25 ~ 05-26	需求追踪矩阵、假设清单、消息事件协议、AgentProfile 草案	每条 PDF 明确需求都有落点；MVP / P2 边界清楚；消息结构固定	范围蔓延、团队各写各的	第一天就固化 requirements-traceability.md；增需求只能替换，不能叠加
阶段二 IM 骨架与运行时基础	05-27 ~ 05-29	会话列表、单聊页、WebSocket、DB/Storage、worktree/sandbox 骨架、Adapter #1	能创建会话、发消息、刷新不丢历史；单个 Agent 可流式回复	UI 漂亮但核心链路不通；运行时隔离太晚做	前端先用 mock + 协议开发；运行时和 worktree 第 5 天必须出原型
阶段三 双 Adapter 与群聊编排	06-01 ~ 06-03	Adapter #2、Orchestrator、群聊分发、失败降级、汇总卡片	复杂任务能分配给至少 2 个 Agent；失败时能退回主 Agent 汇总	多 Agent 互相重复、token 飙升、冲突改文件	限制并行度与最大 turn；默认群聊中的写操作需要主 Agent 汇总后再执行
阶段四 产物、自建 Agent 与部署预览	06-04 ~ 06-05	自建 Agent、skills/、rules/、代码卡片、网页预览、基础部署卡片	可创建一个用户自定义 Agent；至少支持网页 / 代码 / 文件三类产物	文件权限、MCP 工具越权、预览 XSS、部署误触生产	私有 bucket + RLS；第三方 MCP 默认 approval；Preview/Production 隔离

阶段	日期	核心交付物	验收标准	主要风险	缓解措施
阶段五 联调、文档、视频冻结	06-08 ~ 06-09	技术文档、产品文档、AI 协作记录、3 分钟 Demo 视频、兜底录屏	走通“提需求→群聊协作→产物预览→一次修改→预览发布/导出”	Demo 当天上游 Agent 或网络异常	提前录制兜底演示；准备本地样例仓库；所有关键流程写成脚本

阶段性风险里，最值得提前防的是 **MCP / 工具权限**。MCP 规范和 OpenAI 的 MCP/Connectors 文档都明确强调用户授权、数据隐私、工具安全和 third-party server 风险；OpenCode 也把权限系统做成了正式配置。对 AgentHub 这种平台来说，**工具默认从严、逐步放开**是最稳的原则。³¹

每日产出清单

你可以把下面这张表理解成“12 天备忘录版执行图”。每一行都在强迫团队回答一个问题：**今天结束时，仓库里究竟应该多出什么可验收的东西？**

日次	主要产物名称	格式 / 位置	实现要点	验收标准	估计工时 / 优先级	PDF 映射
D1 05-25	需求追踪矩阵；假设清单；优先级 Backlog	docs/requirements-traceability.md docs/assumptions.md docs/backlog.md	把 IM、群聊、Orchestrator、双 Adapter、自建 Agent、产物预览、P2 条目逐条落成 Must/Should/P2	任意一个 PDF 条目都能追踪到负责人、优先级和预计日次	A 4h / B 8h / C 4h; P0	会话列表、单聊、群聊、上下文、产物内联、主协调器、多 Agent 接入、P2 部署/多（用户提供 PDF，第 1-3 页）

日次	主要产物名称	格式 / 位置	实现要点	验收标准	估计工时 / 优先级	PDF 映射
D2 05-26	事件协议; AgentProfile schema; 线 框图	<code>packages/protocol/src/ events.ts</code> <code>packages/protocol/ src/agent-profile.ts</code> <code>docs/ wireframes/*.png</code>	统一 message / status / artifact / deploy 事件; 定义 Agent 的 model、 permissions、 skills、 memory 字段	前后端都能 基于同一份 schema 对 齐; 线框图 能覆盖核心 页面	A 6h / B 6h / C 6h; P0	统一运 配器 层、自 建 Agent 消息类 型、消 息操作 (用户 提供 PDF, 第 1-3 页)
D3 05-27	IM 壳体; 会 话列表; 聊 天页骨架	<code>apps/web/app/(chat)/*</code> <code>apps/ web/components/sidebar/*</code>	完成会话列 表、新建会 话、聊天主区 域、联系人切 换	能从 Web 页面创建新 会话并进入 聊天页	A 12h / B 4h / C 4h; P0	IM 聊 式交 互、对 话列 表、单 聊模式 (用户 提供 PDF, 第 1-3 页)
D4 05-28	WebSocket 流式链路; 消息落库; 历史回放	<code>services/api/ main.py</code> <code>services/api/ ws.py</code> <code>db/schema.sql</code>	聊天消息流式 返回; 刷新后 保留会话历 史; 支持 status message	发送后能看 到流式输 出; 刷新页 面不丢上下 文	A 6h / B 8h / C 8h; P0	上下文 连续、 多轮迭 代修改 (用户 提供 PDF, 第 1-3 页)

日次	主要产物名称	格式 / 位置	实现要点	验收标准	估计工时 / 优先级	PDF 映射
D5 05-29	worktree 管理器; sandbox 基础; Adapter #1 骨架	<code>services/runtime/worktree_manager.py</code> <code>services/runtime/sandbox/*</code> <code>services/adapters/codex/*</code>	一会话一工作树; 最小 sandbox; 把第一个外部 Agent 接成 <code>start/send/stream/cancel</code> 接口	单聊可真正驱动一个外部 Agent; 各会话工作区互不污染	A 4h / B 10h / C 10h; P0	至少接入 2 个主流 Agent 平台、代码二次编辑基础 (用户提供 PDF, 第 1-3 页)
D6 06-01	Adapter #2; 联系人目录; Agent 状态卡片	<code>services/adapters/opencode/*</code> (若账号就绪可替换/追加 <code>claude/*</code>) <code>apps/web/app/agents/*</code>	接通第二个平台; 将 Agent 显示为联系人并带能力标签	两个平台都能被选择并在 UI 中区分	A 6h / B 10h / C 6h; P0	多 Agent 接入、联系人、能力标签 (用户提供 PDF, 第 1-3 页)
D7 06-02	Orchestrator 初版; 群聊 @; 任务拆解规则	<code>services/orchestrator/graph.py</code> <code>specs/orchestrator.md</code>	支持 @ 指定 Agent; 无 @ 时由主 Agent 判断路由; 返回汇总草稿	一个任务可让 2 个 Agent 依次回复; 主 Agent 能总结合并	A 6h / B 12h / C 4h; P0	群聊模式、主 Agent 协调器 (用户提供 PDF, 第 1-3 页)
D8 06-03	并发/降级/冲突策略; 汇总卡片	<code>services/orchestrator/policies.py</code> <code>apps/web/components/summary-card.tsx</code>	设置最大并行度、失败降级、冲突标记; 汇总结构化输出	某个子 Agent 失败时, 会话不崩; 汇总可读	A 4h / B 12h / C 6h; P0	并行调度、失败降级、代码冲突处理 (用户提供 PDF, 第 1-3 页)

日次	主要产物名称	格式 / 位置	实现要点	验收标准	估计工时 / 优先级	PDF 映射
D9 06-04	自建 Agent 页面; skills/rules 初版; 上下文 pin	apps/web/app/agent-builder/ * skills/*/ SKILL.md rules/*.md	支持创建一个用户 Agent; skills/rules 落盘; 可 pin 关键消息	能新建自建 Agent 并在会话中使用; skills/rules 可被调用或引用	A 8h / B 10h / C 4h; P0	自建 Agent 长期上下文 pin、Spec/skill/rules (用户提供 PDF, 第 1-3 页)
D10 06-05	代码卡片; 网页预览卡片; 文件附件; Monaco 集成	apps/web/components/artifacts/ * packages/ui/monaco/*	内联展示代码、预览 iframe、文件附件; 点击可展开查看/编辑	至少三类产物正常展示, 代码可展开编辑	A 12h / B 4h / C 8h; P0	产物内联、网页预览卡片、文件附件、点击展开全屏 (用户提供 PDF, 第 1-3 页)
D11 06-08	部署状态卡片; 预览 URL / ZIP 导出; 安全清单	services/deploy/* docs/ security-checklist.md	先实现最小预览发布链路; 若真实部署来不及, 就保留 ZIP 导出与模拟状态卡	聊天中能触发“部署”或“导出源码”; 有结构化状态反馈	A 4h / B 6h / C 12h; P1	部署发布为 P2; 若真实部署路径不足则保留演示级最小实现 (用户提供 PDF, 第 1-3 页)

日次	主要产物名称	格式 / 位置	实现要点	验收标准	估计工时 / 优先级	PDF 映射
D12 06-09	产品文档； 技术文档； AI 协作记录；3 分钟 Demo 视频	docs/product-design.md docs/ tech-architecture.md docs/ai- collab-log.md demo/ recording.mp4	固化演示路径、补全架构图、准备失败兜底录像	可按脚本完整演示；文档能支撑答辩问答	A 10h / B 6h / C 8h； P0	交付物要求（用户提供 PDF，第 1-3 页）

如果你想把上面 12 天再压实成一句话，那就是：

前 4 天只做“会话和协议”；中间 4 天只做“双 Adapter + 群聊编排”；最后 4 天只做“产物、自建 Agent、交付包装”。

不要在第 3 天就去碰复杂部署，也不要第 6 天还没定消息协议。这种节奏最不容易翻车。

假设、限制与关键行动

先把不能装作“已经明确”的地方老老实实写出来。PDF 没明确预算、上线环境、必须接哪两个平台、并发验收指标，也没规定必须真实接云部署还是允许演示级预览。因此，下面这些都应该在报告中明确标注为“未指定”。

项目	当前状态	处理建议
预算	未指定	不写死模型和平台成本；统一写为“按实时用量计费”
平台账号	未指定	默认先做 Codex + OpenCode ；若 Anthropic 凭据就绪，再接 Claude Code
是否必须真实部署	未指定	建议先做 Preview / ZIP 导出；Production 真部署仅在时间足够时扩展
多端是否必须交付	PDF 有方向，但未强制完整范围	本期只做 Web 主实现；桌面/移动给架构说明与界面预览
性能指标	未指定	建议自定“单会话 < 2s 首 token、群聊汇总可在演示场景内完成”作为内部目标
Diff / 版本历史深度	P2	本期可只做“基础代码卡片 + 应用补丁”预留接口

还需要提醒一个现实问题：**文件与工具安全必须前置，不要留到最后**。MCP 规范把 tool safety、user consent、data privacy 写成显式原则；OpenAI 的远程 MCP 文档建议默认开启 approval，并对第三方 MCP server 做日志与审查；Claude Code 和 OpenCode 都把 skills、hooks、permissions 做成正式概念。这个题如果做成“任何 Agent 都能随便跑 bash、随便推部署、随便读桶”，答辩时会很难解释。 ³²

最后给出一份**非常具体的下一步行动表**。这不是“以后再说”的建议，而是你们明天就可以照着开的任务单。

优先级	行动项	建议负责人
P0	在仓库中先建 <code>docs/requirements-traceability.md</code> 、 <code>packages/protocol/</code> 、 <code>skills/</code> 、 <code>rules/</code> 四个位置，把题目从“口头理解”变成“文件化约束”	B 主责，A/C 参与
P0	先做消息事件协议，再做 UI，不要反过来	B 主责
P0	先确定默认接入的两个平台是 Codex + OpenCode 还是 Codex + Claude Code ；如果账号不稳，立刻切回 OpenCode	B/C 联合
P0	第 5 个工作日前，必须做出“一会话一 worktree + 单聊流式回复”的最小演示	C 主责，B 配合
P1	第 10 个工作日前，至少把网页预览、代码卡片、文件附件三类产物卡片做出来，再考虑更复杂的 Diff 和部署	A 主责，C 配合

一句话收尾：这题最稳的赢法，不是做得最花，而是把“多 Agent 群聊协作”这条主链路做得又稳又清楚。先把 Web 主链路打透，架构讲明白，规范沉淀下来，分数自然就上来了。压力不要给太大，先把最关键的几件事做好。

1 7 12 <https://docs.langchain.com/oss/python/langgraph/overview>

<https://docs.langchain.com/oss/python/langgraph/overview>

2 8 <https://openai.github.io/openai-agents-python/>

<https://openai.github.io/openai-agents-python/>

3 <https://arxiv.org/abs/2210.03629>

<https://arxiv.org/abs/2210.03629>

4 18 29 <https://code.claude.com/docs/zh-CN/overview>

<https://code.claude.com/docs/zh-CN/overview>

5 <https://docs.langchain.com/oss/python/langgraph/workflows-agents>

<https://docs.langchain.com/oss/python/langgraph/workflows-agents>

6 <https://vercel.com/pricing>

<https://vercel.com/pricing>

9 25 <https://github.com/microsoft/agent-framework>

<https://github.com/microsoft/agent-framework>

10 <https://github.com/vercel/chatbot>

<https://github.com/vercel/chatbot>

11 <https://fastapi.tiangolo.com/zh/advanced/websockets/>

<https://fastapi.tiangolo.com/zh/advanced/websockets/>

13 23 <https://github.com/BerriAI/litellm>

<https://github.com/BerriAI/litellm>

14 <https://developers.openai.com/codex/concepts/subagents>

<https://developers.openai.com/codex/concepts/subagents>

15 <https://git-scm.cn/docs/git-worktree>

<https://git-scm.cn/docs/git-worktree>

16 19 <https://supabase.com/docs/guides/storage/security/access-control>
<https://supabase.com/docs/guides/storage/security/access-control>

17 28 <https://github.com/microsoft/monaco-editor>
<https://github.com/microsoft/monaco-editor>

20 <https://opencode.ai/docs/agents/>
<https://opencode.ai/docs/agents/>

21 <https://vercel.com/docs/deployments>
<https://vercel.com/docs/deployments>

22 <https://github.com/langchain-ai/langgraph>
<https://github.com/langchain-ai/langgraph>

24 <https://github.com/openai/openai-agents-python>
<https://github.com/openai/openai-agents-python>

26 <https://github.com/anomalyco/opencode>
<https://github.com/anomalyco/opencode>

27 <https://github.com/OpenHands/OpenHands>
<https://github.com/OpenHands/OpenHands>

30 31 32 <https://modelcontextprotocol.io/specification/2025-03-26>
<https://modelcontextprotocol.io/specification/2025-03-26>